

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

Старший преподаватель

должность, уч. степень, звание

подпись, дата

С.Ю. Гуков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №7

Телефонная книга

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

1 Цель работы

Цель данной лабораторной работы заключается в разработке алгоритма, реализующего структуру данных под названием "телефонная книга", которая эффективно обрабатывает запросы на добавление, удаление и поиск контактов. Данная структура данных должна поддерживать операции добавления записи с указанием номера и имени, удаления записи по номеру, а также поиска имени по указанному номеру.

В рамках лабораторной работы необходимо обеспечить корректную обработку входных данных, минимизировать временные затраты на выполнение операций, а также дополнительно реализовать ограничения и проверки, такие как лимит на количество контактов и защита от дублирования имён. Успешное выполнение задачи позволит изучить основные принципы работы с таблицами прямой адресации, оптимизацию операций поиска и добавления, а также подходы к обработке ошибок и ввода данных.

2 Задание

В данной лабораторной работе требуется разработать программу, реализующую структуру данных «телефонная книга», которая обрабатывает запросы добавления, удаления и поиска контактов. Входными данными является последовательность команд, включающая команды добавления записи с номером и именем, удаления записи по номеру и поиска имени по указанному номеру. Задача состоит в том, чтобы обеспечить эффективное выполнение этих операций с учётом ограничений: номер телефона должен содержать не более семи цифр, а имена не могут превышать пятнадцати символов и должны состоять только из латинских букв.

Программа должна корректно обрабатывать все запросы и выдавать соответствующие ответы. Для запроса добавления запись создаётся или обновляется, если номер уже существует. Для запроса удаления запись удаляется, если номер найден, иначе запрос игнорируется. Для запроса поиска программа должна выводить имя, если запись найдена, или сообщение «not found», если номер отсутствует в телефонной книге. Также реализованы дополнительные функциональности: ограничение на количество контактов в телефонной книге, которое задаётся перед началом работы программы, и защита от дублирования имён. Это ограничение гарантирует, что одно имя может быть связано только с одним номером. Программа должна быть устойчива к некорректным вводам и сообщать об ошибках в случае нарушения правил работы с телефонной книгой.

3 Краткое описание хода работы

В ходе разработки данной лабораторной работы была реализована программа, представляющая собой телефонную книгу, которая обрабатывает запросы добавления, удаления и поиска контактов. Основная задача заключалась в создании структуры данных, способной эффективно управлять контактами и обеспечивать корректность работы при выполнении операций.

Для реализации использовалась структура с прямой адресацией, что позволило минимизировать время поиска, добавления и удаления записей. Особое внимание было уделено обработке ошибок ввода и добавлению защитных механизмов. В частности, реализована проверка формата телефонных номеров, ограничение длины имени и исключение дублирования имён и номеров в телефонной книге. Если пользователь пытался добавить имя, уже связанное с другим номером, программа фиксировала это как ошибку и игнорировала операцию.

Дополнительно в программу была внедрена функциональность для ограничения максимального числа контактов. Перед началом работы пользователю предлагалось указать лимит записей, и при его достижении добавление новых контактов блокировалось. Также реализована обработка всех возможных ошибок, таких как некорректный формат команд, отсутствие обязательных аргументов или превышение заданных ограничений. Программа отслеживала все команды и обеспечивала обратную связь для пользователя в виде информативных сообщений об ошибках и успешных действиях.

Таким образом, разработанный алгоритм и структура данных обеспечивают корректную, надёжную и эффективную работу телефонной книги в заданных условиях.

4 Исходный код

Ниже предоставлен исходный код моей программы с комментариями основных блоков:

```
const readline = require('readline');
```

```
class PhoneBook {
```

```
  constructor(limit) {
```

```
    this.contacts = new Map(); // Хранит номера и имена
```

```
    this.limit = limit; // Максимальное количество контактов
```

```
  }
```

```
  // Добавить контакт
```

```
  add(number, name) {
```

```
    if (!this.isValidNumber(number)) {
```

```
      console.log("Ошибка: неверный формат номера. Номер должен  
содержать от 1 до 7 цифр и не начинаться с нуля.");
```

```
      return;
```

```
    }
```

```
    if (this.contacts.size >= this.limit) {
```

```
      console.log("Ошибка: превышен лимит телефонной книги.");
```

```
      return;
```

```
    }
```

```
    const contactName = name.trim() === "" ? "Не названный контакт" : name;
```

```
  // Проверка на дублирующиеся номера
```

```
  if (this.contacts.has(number)) {
```

```
    console.log(`Ошибка: номер ${number} уже существует с именем  
"${this.contacts.get(number)}".`);
```

```
    return;
```

```
  }
```

```

// Проверка на дублирующиеся имена
for (const [num, existingName] of this.contacts.entries()) {
  if (existingName === contactName) {
    console.log(`Ошибка: имя "${contactName}" уже связано с номером
${num}.`);
    return;
  }
}

this.contacts.set(number, contactName);
console.log(`Добавлено: ${number} -> ${contactName}`);
}

// Удалить контакт
del(number) {
  if (!this.isValidNumber(number)) {
    console.log("Ошибка: неверный формат номера. Номер должен
содержать от 1 до 7 цифр и не начинаться с нуля.");
    return;
  }
  if (this.contacts.has(number)) {
    this.contacts.delete(number);
    console.log(`Удалено: ${number}`);
  } else {
    console.log("Ошибка: номер не найден.");
  }
}

// Найти контакт

```

```

find(number) {
  if (!this.isValidNumber(number)) {
    console.log("Ошибка: неверный формат номера. Номер должен
содержать от 1 до 7 цифр и не начинаться с нуля.");
    return;
  }
  if (this.contacts.has(number)) {
    console.log(this.contacts.get(number));
  } else {
    console.log("Не найдено");
  }
}

```

```

// Проверка корректности номера
isValidNumber(number) {
  return /^[1-9][0-9]{0,6}$/.test(number);
}

```

```

// Настройка ввода
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

```

```

let phoneBook;

```

```

// Запрос лимита перед началом работы
rl.question("Введите максимальное количество контактов: ", (limit) => {
  if (!/^\d+$/.test(limit) || parseInt(limit) <= 0) {

```

```

    console.log("Ошибка: лимит должен быть положительным числом.");
    rl.close();
    return;
}

phoneBook = new PhoneBook(parseInt(limit));
console.log("Телефонная книга готова к работе.");
console.log("Введите команды (add <номер> <имя>, del <номер>, find <номер>, или exit для выхода):");

rl.setPrompt('Введите команду> ');
rl.prompt();

rl.on('line', (line) => {
    const [operation, number, ...nameParts] = line.trim().split(" ");
    const name = nameParts.join(" ");

    switch (operation) {
        case "add":
            if (!number) {
                console.log("Ошибка: номер не указан для команды add.");
                break;
            }
            phoneBook.add(number, name);
            break;
        case "del":
            if (!number) {
                console.log("Ошибка: номер не указан для команды del.");
                break;
            }
    }
}

```



```

        phoneBook.del(number);
        break;
    case "find":
        if (!number) {
            console.log("Ошибка: номер не указан для команды find.");
            break;
        }
        phoneBook.find(number);
        break;
    case "exit":
        rl.close();
        return;
    default:
        console.log("Ошибка: неизвестная команда.");
    }

    rl.prompt();
}).on('close', () => {
    console.log("До свидания!");
    process.exit(0);
});
});

```

5 Результат работы с примерами

В следующих скриншотах будут предоставлены результаты тестирования программы

```
Введите максимальное количество контактов: 2
Телефонная книга готова к работе.
Введите команды (add <номер> <имя>, del <номер>, find <номер>, или exit для выхода):
Введите команду> add Max
Ошибка: неверный формат номера. Номер должен содержать от 1 до 7 цифр и не начинаться с нуля.
Введите команду> add 123 Max
Добавлено: 123 -> Max
Введите команду> add 321 Max
Ошибка: имя "Max" уже связано с номером 123.
Введите команду> add 123 Kyle
Ошибка: номер 123 уже существует с именем "Max".
Введите команду> add 432 Hille
Добавлено: 432 -> Hille
Введите команду> add 65341 Gie
Ошибка: превышен лимит телефонной книги.
Введите команду> del 123
Удалено: 123
Введите команду> find 123
Не найдено
Введите команду> find 432
Hille
Введите команду> exit
До свидания!
```

Рисунок 1 – пример всего функционала

6 Выводы

В результате выполнения лабораторной работы была успешно разработана программа, реализующая функционал телефонной книги, способной обрабатывать запросы добавления, удаления и поиска контактов. Программа учитывает заданные ограничения на формат телефонных номеров и имён, а также обеспечивает корректное выполнение операций с учётом дополнительных требований. Реализация включала механизмы защиты от дублирования имён и контроль за количеством контактов, что позволило создать надёжную и удобную в использовании структуру данных.

В ходе тестирования программа показала свою работоспособность при различных сценариях: добавлении новых записей, удалении существующих контактов, поиске по номеру, а также обработке ошибок ввода. Реализация продемонстрировала устойчивость к некорректным данным, включая ввод неверных форматов номеров или превышение лимита телефонной книги. Это свидетельствует о гибкости и универсальности программы в работе с данными.

Полученные результаты подтверждают, что разработанная телефонная книга способна эффективно управлять контактами и обеспечивать корректность данных даже в условиях ограничений. Работа над лабораторной позволила углубить знания о структуре данных с прямой адресацией, обработке исключений и реализации проверок ввода. Этот опыт способствует развитию навыков программирования и анализа данных, что важно для создания более сложных приложений и алгоритмов, требующих работы с большими объёмами информации. Программа может служить базой для дальнейших улучшений и добавления новых функций, таких как работа с группами контактов или расширенный поиск.